



Storage for CKA

Persistent Volumes and Claims

Introduction to Kubernetes Storage

Hello, and welcome to the course Storage for CKA. My name is Antonio Piedra, and I'll be your guide throughout this course. Storage is one of the key domains in the CKA exam, accounting for a full 10% of the test, so it's critical to get it right. In this course, we are going to start covering what are Persistent Volumes and Claims, why they are crucial for stateful applications in Kubernetes. Next, we will talk about access modes, for example, read-only or read-write, and also we'll dive into reclaim policies for deciding what to do with the data after the Persistent Volume is deleted. After that, we will cover what the storage classes are and how they can be used for dynamic storage provisioning. And finally, I will share some tips to help you nail this part of the CKA exam. Let's get started. Why is storage such an important topic in Kubernetes? It comes down to the nature of containers themselves. By design, containers are ephemeral. This means that when a pod is deleted, crashes, or gets rescheduled to a different node, any data that was saved inside the container's writeable layer is lost forever. This is perfectly fine for stateless applications. Think of our server frontend. If a pod running NGINX crashes, Kubernetes can spin up a new identical replacement. Since it doesn't store any unique data, the application continues to run without any data loss. But what about for stateful applications? Consider a database. A database is useless if it loses all the data every time a pod is restarted. It must maintain its states, the data, across restarts, failures and updates. This is where Kubernetes storage comes in. The entire purpose of the Kubernetes storage system is to provide a stable and persistent place for your stateful workflows completely independent of the pod's lifecycle. To solve this challenge, Kubernetes offers a powerful and flexible set of storage resources. We are going to dive into each of these in the upcoming modules, but let me give you a brief overview of the key players now. At the most basic layer, we have volumes. A Kubernetes volume is simply a directory that is made accessible to the containers within a pod. The crucial part is that its lifecycle is tied to the pod. So if the pod is deleted, the volume is also destroyed. This is great for sharing data between containers in the same pod, but it doesn't solve our data persistence problem. For that, we need to decouple the storage lifecycle from the pod lifecycle. Kubernetes does this with two main objects, Persistent Volumes, PVs. A Persistent Volume is a piece of storage in the cluster. Think of it as a resource in the cluster, just like a node or a CPU. It's provisioned by a cluster administrator or also by a storage class and has a lifecycle that is completely independent of any pod. PVs are implemented using plugins for various storage backends, like NFS, iSCSI, or cloud provider storage, like Google Persistent Disk or



AWS Elastic Block Storage. We also have Persistent Volume Claims, PVCs. A Persistent Volume Claim is a request for a storage by a user or an application. A developer can say I need 5 GB of storage that can be written to without needing to know any details about the underlying storage infrastructure. Kubernetes then matches this claim to a suitable Persistent Volume. Finally, to automate all of these, we have StorageClasses. A StorageClass allows administrator to define the different classes of storage that they offer within the cluster. A class could define performance tiers, like a slow HDD or fast SSD or backup policies. When a PVC requests a certain class, Kubernetes can dynamically provision a brand-new Persistent Volume for just that claim. But how does the binding between the PVC and PV happen? A Persistent Volume Claim, or PVC, is a request for a storage made by a user. When you are deploying an application, you create a PVC manifest that says I need 10 GB of storage and I need to be able to write into it. You don't need to know anything about the underlying storage infrastructure. The Kubernetes control plane then acts as a matchmaker. It looks at your PVC request and finds any available PV in the cluster that meets those requirements. Once it finds a match, it simply binds the PV to the PVC, and this is a 1-to-1 mapping. Due to this process, a Persistent Volume can be in one of few phases. Available; it's free and is not yet bound to any claim. Bound; it has been successfully matched and is in use by a PVC. Released; the PVC using it has been deleted, but the volume is not yet available for a new claim. And also the last phase is failed, which means that the volume has encountered an error.

Demo: Spin up a Minikube Cluster

For all the demos within this course, I have created a series of files that you can find in the Exercises files tab. You can use them to run these steps in your environment, which will also help you prepare for the exam. In this specific demo, we will create the Kubernetes cluster that we will use throughout the course. There are a couple of things that we need to do before getting started. The first one is download and install Docker, and the other one is install Minikube. Minikube is a very lightweight Kubernetes cluster that you can execute in almost every computer. Once both Docker and Minikube are installed, we will start the Kubernetes cluster. For that, we will execute the minikube start command. Note that I'm specifying the driver and also the number of nodes. In this case, 3. The process of starting minikube may take a while because it needs to download some images. Once the start tab is complete, we will run the kubectl cluster-info to verify that the cluster is up and running. We can see that the Kubernetes control plane is running and also we can see that CoreDNS is running on the cluster. Throughout this course, we will be using a series of namespaces for each of the demos. So the next thing that we are going to do is create the namespaces. We are going to have 3 namespaces for modules 1, 2, and 3, and then we will have 2 namespaces for mock-exam1 and mock-exam2. Those are two example questions that we will do at the end of the course. Once the namespace



has been created, we will execute the `kubectl get namespaces` to verify their creation. And we can see that the five namespaces have been successfully created. Whenever you see the sum, you will realize that for each of the tasks they are providing a command to switch the Kubernetes context. Make sure that as soon as you start working on the task, you execute that command. Otherwise, you may end up deleting resources or creating resources in other namespaces. With this set of commands, we will be creating the Kubernetes context that we are going to be using throughout the course. Once we have successfully created the context, we are going to run the `kubectl config get-contexts` to list down the context. And we can see that the contexts have been also successfully created. In the exam, you will be provided a command like the one that we are going to execute now, `kubectl config use-context`, and the name of the context. In this case, `module1-context`. We will run it. And then if we run the `kubectl config current-context`, we will see that the current-context has changed to `module1-context`. Another thing that we are going to be using in this course is using storage classes. One of the storage classes actually needs a `ClusterRoleBinding` to be able to create resources. So we are going to grant the necessary permissions to the storage-provisioner service account. And that should be all for the cluster setup. Whenever you are done working with your cluster locally, you will need to run `minikube stop` to stop the cluster, and then `minikube delete --purge` to remove the cluster completely from your environment.

Demo: Create a PV and PVC

Now that the cluster is up and running, let's look at a basic example of Persistent Volumes and Persistent Volume Claims. The first thing that we are going to do is switch to the module 1 context. Get familiar with this step because you will need to do this in each of the exercises in the exam. Let's now review the Persistent Volume definition that I have prepared. This YAML file defines a Persistent Volume resource called `my-pv`. The capacity of the Persistent Volume is 2 GB. The `accessModes` is `ReadWriteOnce`, and the `hostPath` is `data /my-pv`. We will look at the different type of `accessModes` later on, but for now, let's focus on the `hostPath`. This points to a directory in the Kubernetes node. Now we will execute the `kubectl apply` command to create the Persistent Volume. And after that, we will execute the `kubectl get pv` command to ensure that the Persistent Volume has been successfully created. We can see that the status is set to `available` and the claim is still empty. Now, let's create a Persistent Volume Claim. I have already prepared one called `my-pvc` where we are requesting 2 GB of storage. We are not setting any `storageClassName` because we want to disable dynamic provisioning. We will look at what dynamic provisioning is and how to enable it later on in the course. Now, let's run the `kubectl apply` command to create the PVC. And after that, we will execute the `kubectl get pvc` command, and we can see that the status is set to `Bound`. Now, we will make sure that the PVC has been bound to the correct Persistent Volume. For that, we will run `kubectl get pv`,



and we can see that the PV status is now also Bound and that the claim is set to module1, which is the namespace, /my-pvc, which is the name of the Persistent Volume Claim.

Demo: Deploy a Stateful Application

We are now going to create a pod that makes use of our Persistent Volume Claim. Let's start by reviewing the pod definition. Here we are defining a pod called storage-test-pod, which uses the nginx image. The pod is using the my-pvc storage claim and is setting /usr/share/nginx/html as the mountPath. Let's now create the pod with the kubectl apply command. Now let's wait for the pod to be ready. Now that the pod is running, I will press Ctrl+C, and now we are going to execute the kubectl exec command to include 'Hello from system storage!' within the index.html page of our nginx web server. After that, we will verify that the file was created by using the cat command. As we can see, it's printing 'Hello from persistent storage!' Now let's simulate a failure of a pod by deleting it. We will use the kubectl delete pod command. Deleting the pod should not have deleted the Persistent Volume Claim or Persistent Volume, so let's make sure that they are still there. As we can see, they are both still there and they are bound one to each other. Now let's recreate the pod with the kubectl apply command. Let's try to cat the file again, and we can see that the file is still there. So, we have essentially created a stateful application. No matter how many times the po is restarted, the information will be preserved.

Access Modes and Reclaim Policies

Volume Types and Access Modes

We got our hands dirty, creating Persistent Volumes and Persistent Volume Claims and proof that they allow our data to survive when a pod is deleted, but this raises two important questions. How do we control how multiple pods can access the same volume, and what happens to the data on the volume when we delete the claim? In this module, access modes and reclaim policies will answer both these questions. Let's briefly touch on volume types. Kubernetes supports many different kinds of volumes from single or node storage, like hostPath, to network file systems, like NFS, or cloud provider-specific storage, like awsElasticBlockStorage, or gcePersistentDisk. The most important thing to know is that the underlying storage technology dictates which access modes are supported. A block storage volume from cloud provider can often only be attached to a single node at a time, which limits its access options. This brings us to volume access mode. Access modes define the read-write permission and sharing capabilities of a volume. There are three main modes you must know



for the CKA exam. `ReadWriteOnce`, the most common and widely supported mode. It means the volume can be mounted as read-write by a single node. Be careful with the wording here. It doesn't mean a single pod. You can have multiple pods on the very same node using a `ReadWriteOnce` volume at the same time. The next mode is `ReadOnlyMany`. This mode allows the volume to be mounted as a read-only by many nodes at the same time. This is perfect for sharing data that doesn't change often, like configuration files or a static website access across your entire cluster. The last mode is `ReadWriteMany`. This mode allows the volume to be mounted as read-write by many nodes simultaneously. This is the most flexible and powerful mode, but it requires a storage backend that can support it, like NFS or other distributed file systems.

Demo: Volume Access Modes

In this demo, we are going to see how using a `ReadWriteOnce` access mode in our Persistent Volume prevents us from using the volume across multiple nodes within the cluster. Let's start by switching the context to `module2`. Then we will review the Persistent Volume Claim definition. And here we are defining a Persistent Volume Claim called `rwo-pvc`. In this case, we are using the `csi-hostpath` storage class to automatically create the Persistent Volume for us. We will check how dynamic provisioning works later on in the course. We are also defining the access mode, in this case, `ReadWriteOnce`, and also requesting 1 GB of _____. Let's now create the PVC resource with the `kubectl apply` command. After running this, the Persistent Volume should also be created. So let's verify that. We can see that the Persistent Volume Claim has been created and also the Persistent Volume, and the status of both of them is set to `Bound`. Now let's see in what node the Persistent Volume was actually created. The Persistent Volume was created in the second node. So now we will create a pod that runs on that very same node and uses the Persistent Volume Claim. Let's review the definition. In line number 7, we are using the node name where the pod will run. We are also defining the Persistent Volume Claim. We'll create the pod using the `kubectl apply` command. After creating the pod, we will run `kubectl get pod` to verify that the pod is up and running, and we can see that the status is reported as `running`. Now we will try to create a second pod called `pod-b` that runs on a different node. In this case, it's going to run on node number 3 and also will try to mount the same Persistent Volume Claim. Let's try to create the pod. We can see that the pod was successfully created and we verify the status with the `kubectl get pod` command. We can see that the pod is stuck in `ContainerCreating` state. Let's investigate why that's happening using the `kubectl describe` command. In the event we can see the `FailedMount`, so the node was unable to mount the PVC. If the volume was `ReadWriteMany`, both pods will start successfully, even on different nodes. Let's now perform some cleanup. We will start by deleting the pods. And finally, we will delete the Persistent volume claim which will also end up deleting the Persistent Volume.



Volume Reclaim Policies

Now, let's tackle our second question. What happens to the data when the PVC is deleted? This is controlled by the volume reclaim policy. This policy tells Kubernetes what to do with the Persistent Volume after its claim is released. There are three policies. Retain, which is the default and safest policy. When the PVC is deleted, the PV is not deleted. It enters a released state. The data on the underlying storage volume remains untouched. A cluster administrator must then manually inspect the volume, back up the data if needed, and then clean up the volume and storage. The second policy is delete. When a PVC is deleted, the Persistent Volume object is also deleted. Kubernetes instruct the storage backend to delete the underlying storage asset as well. This means that your AWS or GCE disk will be gone forever. This is very useful for temporarily or scratch data where you want automatic cleanup to happen. And the last policy is recycle. This policy is deprecated. It used to perform a basic `rm -rf` on the volume's contents to clean it up for reuse. You should not use it, but it's good to know that it once existed. Modern dynamic provisioning with storage classes has replaced this functionality.

Demo: Volume Reclaim Policies

In this demo, we're going to check how the reclaim policies affect the Persistent Volumes when their claims are deleted. First, we are going to create two Persistent Volumes, one called `pv-retain` and the other one called `pv-delete`. The definition of the two persistent volumes is pretty much the same. The only differences are line number 5, where we are defining the name, and also line number 11, where we are defining the Persistent Volume Reclaim policy. In one of them is `delete`, in the other one we are setting `retain`. Let's now create the two Persistent Volumes using the `kubectl apply` command. Now, we will also create Persistent Volume Claims. Similarly to the Persistent Volume, our Persistent Volume Claim definitions will be pretty much the same as well. The only difference will be the name. Now, we are also going to execute the `kubectl apply` command to create both of the Persistent Volume Claims. Then we will verify that both the Persistent Volume Claims and the Persistent Volumes are created and bound. As we can see, the status for both the Persistent Volume Claim and also the Persistent Volumes is set to `Bound`. Now we are going to delete both of the PVC using the `kubectl delete pvc` command. After that, we will check the status of the PV with the `kubectl get pv` command, and you will notice that we only have a single Persistent Volume. The `pv-delete` Persistent Volume has been deleted after we removed the Persistent Volume Claim. And this is due to the `delete` reclaimed policy. We still have the `pv-retain` Persistent Volume, and that's because we set `retain` as the reclaim policy. The data remains on the node, and if we want to reuse it again, then it will require intervention. We will also notice that



the status has changed from Bound to Released. Based on your application requirements, in some cases you may choose delete for your Persistent Volume Claim when the data isn't really needed, or you may use the retain reclaim policy for applications like a database for which you need to keep the data. Let's now perform some cleanup of the Persistent Volume that is still there.

Storage Classes and Dynamic Provisioning

Storage Classes in Kubernetes

We'll learn how to manually create and manage Persistent Volumes. While this is a fundamental skill, you can probably see that it doesn't scale. Imagine being a cluster administrator and having to manually create a Persistent Volume every time a developer needs a piece of storage. It would be slow, tedious, and prone to error. This is the problem that our next topic solves. In this module, Storage Classes and Dynamic Provisioning, we learn how to automate the entire process of creating Persistent Volumes. So, what is a StorageClass? A StorageClass is a cluster-level resource that allows an administrator to define different classes or tiers of storage that they offer. You can think of it as a template or a factory for creating Persistent Volumes. For example, you could create a fast SSD class that provisions high performance SSD storage from a cloud provider, and a slow HDD class that provisions cheaper, magnetic storage for backups or archives. This is a sample StorageClass definition. The most important field in a StorageClass definition is the provisioner. This tells Kubernetes which volume plugin to use when it needs to create a Persistent Volume for this class. Every major cloud provider has its own provisioner, like the one that we see in a screen for Google, and there are provisioners for many other storage systems, like NFS. The StorageClass also contains parameters that are passed to the provisioner. For a cloud provider, this is where you would specify the disk type, like pd-standard in this case, and whether the volume should be replicated or not. The next important setting is the reclaim policy. It dictates what happened to the physical storage, the Persistent Volume, and the data when the user deletes the Persistent Volume Claim. As we saw earlier on the course, there are two main options. Delete is the default for dynamic provisioning, and it automatically deletes the Persistent Volume and the underlying cloud or physical storage assets. The data is permanently lost. The other option is retain, which keeps the Persistent Volume in a released state and the underlying storage assets intact, preserving the data. An administrator must manually clean up the resource. There was also a third option, recycle, which is officially deprecated nowadays. The next parameter for the StorageClass resource is allowVolumeExpansion. When set to true, users can increase the size of an existing PersistentVolumeClaim, and the storage provisioner will update the Persistent Volume accordingly. And finally, we have the volumeBindingMode. This is a crucial setting in Kubernetes



StorageClass that dictates when a Persistent Volume Claim should be bound to a Persistent Volume and when the actual storage volume should be provisioned. It has two possible values. Immediate, the Persistent Volume is created just after the PVC is created. And the other option is WaitForFirstConsumer. When this option is set, the Persistent Volume is only provisioned after a pod is scheduled to a node. With StorageClasses, when a user wants storage, they no longer need an admin to create the Persistent Volume first. They can simply create the Persistent Volume Claim and in the manifest specify the StorageClass name they want to use. Kubernetes sees the Persistent Volume Claim, looks up the StorageClass, and invokes the correct provisioner to create a brand-new Persistent Volume on the fly that perfectly matches the user's request. The new Persistent Volume is then automatically bound to the Persistent Volume Claim. This entire automated workflow is known as dynamic provisioning.

Demo: Dynamic Volume Provisioning - Immediate Binding

In this demo, we are going to be using the default storage class to provision a Persistent Volume as soon as the PVC is created. The first thing that we are going to do is switch the context to the module3 context. After that, we will check the available storage classes using the `kubectl get storageclass` command. We have two storage classes, `csi-hostpath-sc`, and also the standard storage class. We can see that next to the standard storage class, we have default between parentheses. This means that if no storage class is defined in your persistent volume, the standard storage class will be used. Note also the reclaim policy for both storage classes, which is set to Delete, which means that as soon as the Persistent Volume Claim is deleted, the Persistent Volume will also be deleted. Also, the `volumeBindingMode` is set to Immediate, so as soon as the PVC is created, the Persistent Volume will also be created. Let's now use the `kubectl describe storage class standard` command to review the storage class configuration. We can see that the `k8s.io/minikube-hostpath` is the provisioner for this storage class. We can see also the `ReclaimPolicy` set to Delete and also the `VolumeBindingMode` set to Immediate. Now we will try to create a Persistent Volume. Let's review the definition. In this YAML file we are defining the `my-immediate-pvc` Persistent Volume Claim and we are not specifying any storage class name, which means that the default one will be used. Let's execute now the `kubectl apply` command to create the Persistent Volume Claim. After that, we will use the `kubectl get pvc` command to check the status of the new resource that we have just created, and we can see that the status is set to Bound. And we can also see the Persistent Volume that was created. Let's also check for other volumes that may exist in the cluster. We will do this by executing the `kubectl get pv` command. We can see that there is only one volume and it's the one that got created when we created the Persistent Volume Claim. So the Persistent Volume has been created automatically. It's bound to our PVC. The size also



matches the one that we set up in the PVC definition, which is 1 GB. We don't need to create a pod or any consumer in order for the Persistent Volume to be created because in the storage class, we are defining Immediate as VolumeBindingMode. So this means that the storage is reserved even if it's not used yet. So what will happen if we delete the Persistent Volume Claim? Let's execute the `kubectl delete pvc` command to delete the Persistent Volume Claim. Ideally, as the reclaim policy was set to Delete, the Persistent Volume will no longer be there either and is no longer there, so the standard storage class is working as expected.

Demo: Dynamic Volume Provisioning - WaitForFirstConsumer Binding

In this demo, we are going to use a storage class with the VolumeBindingMode set to WaitForFirstConsumer. This delays the Persistent Volume provisioning until a pod using the Persistent Volume Claim is created. The first thing that we are going to do is switch the context to the module3 context. Then we will need to create a new storage class that will have the WaitForFirstConsumer binding mode. Let's review the storage class definition. The name for the storage class is defined in line number 5 and is set to delayed-binding-sc. The provisioner is the minikube-hostpath and the VolumeBindingMode is set to WaitForFirstConsumer. We are not defining the reclaim policy, so the default one will be applied, delete. Now we will create the storage class using the `kubectl apply` command. And once the storage class has been created, we will create a Persistent Volume Claim using the new storage class. Let's review the definition. The name of the Persistent Volume Claim is going to be my-delayed-pvc, and in line number 7, we are defining the storageClassName. In this case, delayed-binding-sc. Now, let's create the Persistent Volume Claim using the `kubectl apply` command. After that, we will execute the `kubectl get pvc`. And we see that the status of the PVC remains in Pending. This is because it's not bound to any Persistent Volume yet, and the reason why it's not bound to any Persistent Volume yet is because the Persistent Volume doesn't exist because the storage class has not created it yet. Let's run the `kubectl get pv` to ensure that we don't have any Persistent Volume. No resource is found, so the Persistent Volume is not yet created. Now, to force the creation, we will create a pod that will make use of the Persistent Volume Claim that we have just created. Let's start by reviewing the pod definition. In this pod called delayed-pod, we are defining a volume which uses the Persistent Volume Claim my-delayed-pvc. Let's now use the `kubectl apply` command to create the pod. Now let's check the status of the pod and also the PVC using the `kubectl get` command. We can see that the pod is running and also the Persistent Volume Claim status is now set to Bound, and the volume field also contains data. So this suggests that the Persistent Volume has been created after we created the pod. Let's run the `kubectl get pv` command to ensure that the Persistent Volume has been created. And a new Persistent



Volume has been created for our claim, and the status is set to Bound. The Persistent Volume was created only after the pod was scheduled. This avoids having in your cluster unused volumes, which means that you are managing your resources more efficiently. Also, the volume is created on the node where the pod is scheduled, which leads to better scheduling decisions. Now let's perform some cleanup. Let's start by deleting the pod with the `kubectl delete pod` command. After that, we will delete the Persistent Volume Claim, which will also delete the Persistent Volume as the reclaim policy is set to delete. And finally, we will delete the storage class using the `kubectl delete sc` command.

Exam Preparation

Exam Overview

We have learned why storage is critical in Kubernetes. We talk about the stateful applications and the need of persistent volumes, claims, and storage classes for dynamic provisioning. We also learned how to control the behavior of the volumes with access modes and reclaim policies. You should now have all the knowledge required to pass this part of the CKE exam. The most important thing to know about the CKA exam is that it's a practical hands-on test. You won't be answering multiple choice questions. Instead, you will be working in a live command-line environment to solve real-world problems. The exam will be online and remotely proctored. You will need to connect via a video call with a proctor, identify yourself, and ensure your desk is clear. You will have 2 hours to complete the exam directly from your browser, and there will be between 15 and 20 practical tasks that require working directly in a Kubernetes cluster. The 2 hours can be a little bit tight if you get stuck in one of the questions. To pass the exam, you need to achieve a score of 66% or higher. During the exam, you are allowed to have an additional browser tab open with the official Kubernetes documentation, kubernetes.io/docs/llados and resources under the official GitHub for Kubernetes. I'm pretty sure that you will pass the exam on your first attempt. But to give you some more confidence, the exam registration typically includes one free retake. Let's quickly touch on the domains and competencies included in the CKA exam. Ten percent goes towards the storage domain, 30% towards troubleshooting tasks, 15% for workloads and scheduling, 25% to test your knowledge on Kubernetes architecture, installation, and configuration, and the remaining 20% goes towards services and networking. The storage domain account for 10% of your total score. You should expect to see 2 or 3 hands-on tasks related to the topic that we have covered. And that includes implement the storage classes and dynamic provisioning, configured volume types, access modes, and reclaim policies, and manage persistent volumes and persistent volume claims.



Demo: Recommended Exam Environment Setup

Once you get access to the exam, I would recommend doing a very basic environment setup that will help you gain some precious seconds. The environment uses the bash shell, so the first thing that you should do is set up an alias for kubectl. You'll be under a lot of pressure, and spending time in typing and misspelling kubectl is the last thing that you want to do. As soon as you connect to the environment, run `alias=kubectl`, and then you will be able to run things like `k get pod -A` to get the pods across all namespaces. Also, know the short version of the resources. For example, `sc` for storage class. So you can do `k get sc` to get the storage classes in your environment. Another example will be `rs` for getting the replica set. You can check all the available short names using the `kubectl api-resources` command. Notes that not all the resources have short names, but most of the resources that you will be using in the CKE exam have short names, so make sure that you spend some time reviewing this. The other thing that you should do is using kubectl to generate boilerplate YAMLs for you. Let's say that you are required to create a pod named `pod1` running the `nginx` image and with an `emptyDir` volume. So for that you will need to create a manifest file. You can use `k run pod1` and specify the image with the `image` flag, in this case, `nginx`, and then do `dry-run=clients` to not create the resource and then specify the output to be `YAML`. This will give you a very basic manifest file that you can then modify to add the `emptyDir` volume. Another thing to do is exporting the `dry-run` and `output` flags. So you can do `export do="--dry-run=client -o yaml."` And whenever you need to create a pod, you can do `k run pod2` and specify the image, in this case, `busybox` `$do` for dry run and output `yaml`. This will also generate the pod definition and avoid typos. You can also redirect the output to `pod.yaml`. Then you can do the require modification with `me`. And after that, you can run `k apply -f` and the name of the file to create the pod. The last thing that I will recommend is preloading the Kubernetes quick reference page. It contains a lot of commands that you will use throughout the exam. Also, before the exam, make sure that you read the page entirely at least a couple of times. You don't need to know things by heart, just know that they are there and that you can use them as a reference. You are ready to start your exam. Let's look at some example questions related to storage.

Demo: Example Exam Question 1 - Static Provisioning

Let's start with an example exam question. In this case, we need to create a `PersistentVolume` named `task-pv` with a capacity of `200Mi`. It should have an access mode of `ReadWriteOnce` and a reclaim policy of `Retain`. It should use a `hostPath` at `/opt/data`. And once created, we need to create a `PersistentVolumeClaim` named `task-pvc` in the `mock-exam1` namespace that also requests `200Mi` and binds to the `task-pv`. We are also asked to switch context, so



that's the first thing that we are going to do. After that, as we need to create a PersistentVolume of type hostPath, I will search in the Kubernetes documentation for hostpath persistent volume. The first result is configure a pod to use PersistentVolume for storage. That probably has the PersistentVolume and also the PersistentVolumeClaim. So let's open it, and if we scroll down, we will see one of the sections for PersistentVolume. Here we have a PersistentVolume definition with accessMode of ReadWriteOnce, and also we are specifying the hostPath. So I will copy this and start working with it. I'm going to use vim to create a file called pv.yaml. Then I will paste my PersistentVolume definition, and we will start modifying it. The name of the PersistentVolume should be task-pv. We don't need any label, so I will remove that. For the storage class, we need the static provisioning, so I will specify an empty value. For the storage, we need 200Mi, so I will define that. For the accessMode, we need ReadWriteOnce, so we don't need to do any changes there. And for the hostPath, we need /opt/data. That should be all for the PersistentVolume. So I'm going to save the file and then create the PersistentVolume with the kubectl apply -f command. And the PersistentVolume has been created. Let's verify its status. Kubectl get pv, and we see that the PersistentVolume has been created and the status is set to Available. Now, let's create the PersistentVolumeClaim. If we go back to the Kubernetes documentation and scroll down, we see that it's trying to create the PersistentVolumeClaim as well. So I will copy it. And back to the terminal, I will create the pvc.yaml file with vim. I will paste the content, and we will start modifying what we need. So, the name of the PVC it should be task-pvc. We want a static provisioning, so we shouldn't specify any storage class. For the accessMode, it should be ReadWriteOnce. And for the storage, it should be 200Mi. This should be ready. So I'm going to save this, and then I will create it with the kubectl apply -f command. The PersistentVolumeClaim has also been created, so let's now make sure that it has been successfully bound to the PersistentVolume. We will execute kubectl get pv and also pvc. And we can see that the status of the PV is set to Bound, and we can also see the claim associated to it. In the PersistentVolumeClaim resource, we see that the status is set to Bound, and the volume is task-pv. We also see that the capacity is 200Mi, and the accessMode is set to ReadWriteOnce. So this task is now complete.

Demo: Example Exam Questions 2 - Dynamic Provisioning

Let's check another example question. In this case, we need to create a PersistentVolume named dynamic-pvc in the mockexam2 namespace. We need to use the cluster's default storage class, and the PVC must request 50Mi of storage. Then we need to create a pod named data-pod that uses the busybox latest image and mounts the PVC at the path/data. We are also asked to switch the context, so that's the first thing that we are going to do. As we need to create a PersistentVolumeClaim and mount it into a pod, we will look for that in



the Kubernetes documentation. Remember that in the CKE exam, they are not testing your memory. They are testing that you know how to apply the content. So now, let's jump into the Kubernetes docs page and search for persistent volumes. The first result is for configuring a pod to use a PersistentVolume for storage. So let's look at it. First is creating a PersistentVolume, which we don't need because we want to use dynamic provisioning, and then it's defining the PersistentVolumeClaim. This we can use and modify as per our needs. So I'm going to copy this, and then I'm going to use vim to create the dynamic-pvc.yaml file. I'm going to paste the content. And I'm not going to specify any storage class because we want to use the default one. The accessMode is not specified, so we will just leave ReadWriteOnce. And for the storage, we need to have 50Mi. We also need to change the name of the PersistentVolume to dynamic-pvc. Then I'm going to save the file. And using the kubectl apply -f command, I will create the PersistentVolumeClaim. Now we need to make sure that a volume has been created for the PersistentVolumeClaim, so let's execute kubectl get pv and also pvc. We see that the PersistentVolumeClaim has been created, and it's already associated to a volume. The status is also Bound, and the capacity is 50Mi. So the PVC creation was successful. Now let's create the pod. Let's go back to the Kubernetes documentation. And if we keep scrolling, we see a section for creating the pod. I'm going to use that as a reference. So I'm going to copy it, and then I'm going to create a file called data-pod.yaml, and I'm going to start modifying it as per our needs. So the name of the pod is going to be data-pod. The name of the volume is going to be test. And it's going to be associated with the PersistentVolumeClaim that we created before, dynamic-pvc. The container name is going to be test-container, and the image is going to be busybox:latest. We don't need to have any pods, so I'm going to remove it. And in the volumeMounts section, we are going to specify the mountPath to be /data, and the name of the volume is going to be test. I'm going to save this. And then we're going to create the pod using the kubectl apply -f data-pod command. We will make sure that it was able to start executing the kubectl get pods. And we see that it's not able to start. So, what is happening? Let's troubleshoot. We can execute k describe pod, and we see that the container is starting, but it just crashes. So this is because the busybox image doesn't do anything on its own. We need to specify a command for the container to use when it starts. Let's open again the pod definition. Now under container, we are going to specify the command. In our case, it's going to be /bin/sh. And for the arguments, we will pass the sleep command, which will make the container to sleep for 5 minutes. Save the file and then delete the pod with the kubectl delete pod data pod. And then we're going to click created. Now, let's check that it's working as expected, and the pod is successfully running now. So, this task is also complete.

Exam Tips

Here are my top tips for the exam. Set up an alias for kubectl and also for -dry-



run=client and -o yaml. They will save you a massive amount of time, and it helps avoid typos. Learn and use the short name of the resource types. You want to spend the minimum time possible typing. Always check your context. The exam will have multiple namespaces. Before starting any task, make sure that you are working on the correct one using the `kubectl config use context` command given in the prompt. Use the official documentation during the exam, especially the quick reference page. Don't be afraid to use it to look up the exact syntax for a resource you are not 100% sure about. Remember that only one additional tab is allowed. Spend some time playing with the Kubernetes docs page ahead of the exam. Knowing how to quickly navigate and search this documentation is crucial for success. And finally, manage your time efficiently. If you are stuck on a question, flag it and move on. It's better to complete five easy tasks than to get stuck on a difficult one. At the end of the day, you don't need to solve all the tasks to pass the exam, just 66% of them. And that brings us to the end of the course. We have covered the full spectrum of Kubernetes storage, and you are fully prepared for the storage portion of the CKA exam. Thank you for joining me, and good luck on your exam!